MeetBrief AI — Complete Project Specification and Planning Brief

1. Your Role and Objective

Act as a senior full-stack software engineer and technical mentor experienced in LLM application development, React, Node.js, Express, MongoDB, document processing, speech-to-text systems, and conversational AI.

I am a computer science engineering student developing an LLM implementation project for a college assessment. I need a detailed, practical implementation plan for the project described below.

Your first task is to create the implementation plan, not to start coding immediately.

The plan must be beginner-friendly, technically accurate, implementation-oriented, and realistic for a small student team. Explain important architectural decisions and why they are appropriate. Avoid unnecessary technologies, infrastructure, and features that increase complexity without providing meaningful value.

---

2. Project Overview

Project name: MeetBrief AI

Project title: MeetBrief AI – An LLM-Powered Meeting Minutes and Action-Item Generator

Project category: LLM-powered web application with document processing and conversational question-answering capabilities.

Project description

MeetBrief AI is an LLM-powered meeting assistant that converts voice recordings, meeting transcripts, and PDF meeting notes into structured meeting minutes.

The application automatically generates concise summaries, extracts important decisions, identifies action items, records assigned responsibilities and deadlines when explicitly mentioned, and highlights unresolved questions.

After receiving the initial summary, users can continue asking questions about the uploaded meeting content through a conversational chat interface. The LLM must use the relevant meeting content and conversation context to answer follow-up questions.

The application should have an interface inspired by ChatGPT, with a simple sidebar containing previous conversations and a New Chat option. Selecting a previous conversation should restore the exact conversation history, including the user's original messages and the assistant's previously generated responses.

There will be no login or registration system. Conversation history will be associated with an anonymous browser identifier.

Primary objective

Build a functional application that demonstrates how an existing LLM can be integrated into a useful software product, rather than training a model from scratch.

The project should demonstrate:

- LLM API integration and prompt engineering.
- Text extraction from supported files.
- Audio transcription.
- Structured information extraction.
- Conversational follow-up questions using meeting context.
- Persistent chat history.
- A usable full-stack web interface.
- Basic reliability, error handling, and evaluation.

---

3. Target Users and Use Cases

The application should be useful for students, project teams, meeting participants, and small teams that need to document discussions efficiently.

Typical use cases include:

1. A student uploads a recording of a college project meeting and receives organized meeting minutes.
2. A team member uploads a PDF containing meeting notes and extracts decisions and assigned tasks.
3. A user pastes an existing transcript and asks the LLM to summarize it.
4. A user asks, "What tasks were assigned to Ravi?" after the initial summary.
5. A user returns to a previous conversation and continues discussing the same meeting.
6. A user reviews the generated minutes and downloads them for documentation.

The application is not intended to replace a human's final review of official meeting minutes.

---

4. Functional Requirements

4.1 Multiple input methods

Users must be able to supply meeting content through the following methods:

A. Audio recordings

Users can upload recorded meetings for transcription and subsequent analysis.

Choose an appropriate speech-to-text service or model. The implementation plan should specify how the audio will be uploaded, processed, transcribed, and passed to the LLM.

Common audio formats such as MP3, WAV, and M4A may be supported where compatible with the chosen service.

Consider file-size limits, duration limits, processing time, API costs, and error handling.

Live recording and real-time transcription are not required for the initial version. Processing uploaded recordings is sufficient.

B. TXT transcripts

Users can upload `.txt` files containing meeting transcripts.

The backend should validate the file and extract its contents for analysis.

C. PDF transcripts

Users can upload PDFs containing meeting transcripts.

The backend should extract text from ordinary PDFs with selectable text. Scanned PDFs requiring OCR may be supported later as an optional enhancement.

D. PDF meeting notes

Users can upload PDF documents containing informal or structured meeting notes.

The application should process the extracted text in the same general way as a transcript, while recognizing that meeting notes may be incomplete or lack speaker information.

E. Direct text input

Users can paste meeting notes or transcripts directly into the chat input without uploading a file.

All input methods should eventually feed into a common text-processing and meeting-analysis workflow.

The initial interface should clearly display the selected file or the text being submitted and provide understandable error messages for unsupported or invalid files.

4.2 Automatic meeting minutes

After the meeting content has been extracted or transcribed, the backend should send it to the LLM with a predefined system instruction or prompt template.

The LLM should generate the following sections:

1. **Meeting summary:** A concise overview of the discussion.
2. **Key decisions:** Decisions explicitly made during the meeting.
3. **Action items:** Tasks or commitments mentioned in the meeting.
4. **Assigned responsibilities:** The people responsible for the tasks, when identifiable.
5. **Deadlines:** Dates or timeframes explicitly mentioned.

6. **Pending issues:** Unresolved questions, unassigned tasks, and matters requiring further discussion.

The output should be structured, preferably using a JSON schema or equivalent structured-output mechanism when supported by the chosen model.

The frontend should display these sections in a readable format rather than exposing raw JSON to users.

Accuracy requirements:

- Do not invent participant names, decisions, deadlines, or commitments.
- Mark missing information as "Not specified" or "Unassigned."
- Distinguish a confirmed decision from a suggestion or an unresolved discussion.
- Avoid presenting uncertain interpretations as established facts.
- Allow users to review and correct the generated results.

If the LLM cannot reliably extract a particular item, it should state that the information is unavailable rather than fabricate it.

4.3 Follow-up questions and conversational interaction

Generating meeting minutes is only the initial step. Users must be able to continue the same conversation by asking further questions.

Example:

User: Uploads meeting notes and requests a summary.

MeetBrief AI: Generates the summary, decisions, action items, responsibilities, and pending issues.

User: "What tasks were assigned to Ahmed?"

MeetBrief AI: Answers using the source meeting content.

User: "What did we decide about the database?"

MeetBrief AI: Retrieves the relevant information from the meeting content and responds.

User: "Were any deadlines specified?"

MeetBrief AI: Lists explicitly mentioned deadlines or states that none were found in the available source.

The implementation must retain access to the extracted meeting text after the initial response.

For each follow-up request, the backend should supply the relevant source content and appropriate recent conversation history to the LLM. It should not depend exclusively on the previous summary, because the summary may omit details present in the original transcript.

The plan should account for context-window limits when processing long meetings. Start with a simple approach suitable for ordinary-length transcripts; consider chunking or retrieval only where necessary.

Responses should be grounded in the uploaded meeting content. If a question cannot be answered from the source, the application should communicate that limitation.

4.4 Conversation history

The application must behave like a basic ChatGPT conversation interface.

Users should be able to:

- Start a new conversation.
- Upload or paste the content for a meeting.
- Receive the initial generated response.
- Ask follow-up questions.
- View their earlier messages and the assistant's earlier responses.
- Select a previous conversation from the sidebar.
- Continue an existing conversation.
- Delete a conversation, if this can be implemented without significantly increasing the initial scope.

Each conversation should represent one meeting or one logically related discussion. A new conversation should not accidentally inherit the previous conversation's meeting context.

Important persistence requirement:

When a user opens an earlier conversation, the application must retrieve and display the previously saved messages in their original order.

It must not call the LLM again merely to regenerate the historical responses. Historical message content should be stored and rendered as originally returned, unless the user explicitly edits or deletes it.

The original meeting context must also remain available for follow-up questions after the conversation is reopened.

The sidebar should show a useful conversation title, preferably generated from the meeting or derived from the first user message. The title should be saved so that it remains consistent when the page is reloaded.

4.5 New Chat behavior

The New Chat button should start an independent conversation.

It should clear the active conversation view and allow the user to provide new meeting content.

A conversation can be saved when the user first submits text or uploads a file. Avoid creating unnecessary empty database records every time the user clicks New Chat.

The exact lifecycle of a new conversation should be explained in the implementation plan.

4.6 Review and export

Users should be able to review the generated meeting minutes.

Provide an option to download the results as a simple text or Markdown file. PDF export is optional.

Editing the generated meeting minutes in the interface is desirable, but it can be treated as a secondary feature if the basic workflow is not yet complete.

Exporting a formatted meeting summary is sufficient; a sophisticated document editor is not required.

---

5. User Interface Requirements

The interface should be inspired by the general layout of ChatGPT, not a pixel-perfect copy of its branding or every feature.

5.1 Overall layout

Use a responsive application layout with two main regions:

A. Sidebar

The sidebar must contain only:

- The MeetBrief AI application name or logo.
- A New Chat button.
- A list of previous conversations.

Conversation entries should be selectable. The currently active conversation should be visually distinguished.

Provide a simple way to delete a conversation if deletion is included in the implementation.

Do not add unnecessary navigation items, such as image generation, plugins, dashboards, administration pages, or unrelated tools.

B. Main chat section

The main section should contain:

- The conversation title or an appropriate initial welcome state.
- User messages.
- Assistant responses.
- Structured meeting-minutes sections.
- File upload controls.
- A text input area.
- A send button.
- A loading indicator while processing.
- Clear error messages.
- A download option for the generated results.

Users should be able to upload a file or paste text through the chat interface. After the file is submitted, the conversation should display an understandable indication of what was uploaded.

Use a clean, minimal visual style inspired by the supplied ChatGPT screenshot. A dark theme is acceptable and may be used as the default. Keep the layout focused on the conversation and avoid unnecessary interface elements.

The chat input should remain easy to access. Long conversations should be scrollable, and new responses should appear at the bottom.

The interface should work on common desktop screen sizes and remain usable on smaller screens.

5.2 Initial state

When the user opens the application for the first time, show a simple welcome message explaining that they can upload a recording, upload a transcript or PDF, or paste meeting notes.

Provide a clear upload control and message input.

Do not display fabricated previous conversations.

5.3 Loading and error states

The interface should communicate when the application is:

- Uploading a file.
- Transcribing audio.
- Extracting PDF text.
- Generating meeting minutes.
- Answering a follow-up question.

Provide useful messages for unsupported formats, empty files, files without extractable text, transcription failures, network problems, and LLM API errors.

Disable or otherwise protect submission controls when necessary to prevent accidental duplicate requests.

---

6. Authentication and User Identity

A login page is not required.

Do not implement account creation, passwords, email verification, social login, or user profile management.

For the initial implementation, generate an anonymous browser identifier and persist it in `localStorage`. Include this identifier in requests that access or modify conversations.

The backend should use that identifier to associate conversations with the originating browser.

This is a convenience mechanism, not strong authentication. A person who obtains another browser's identifier may be able to impersonate that browser. The application must not claim that anonymous browser IDs provide the security of authenticated accounts.

The implementation plan should include appropriate conversation-access checks, input validation, and file handling.

Because meetings may contain sensitive information, advise using non-sensitive sample transcripts for classroom demonstrations.

---

7. Recommended Technology Stack

The following technologies are proposed. Use them as the default unless there is a clear technical reason to choose an alternative.

Frontend

- React.
- JavaScript, unless TypeScript provides a clear advantage without excessive complexity.
- Standard CSS or a lightweight styling solution.
- A chat layout inspired by ChatGPT.

Backend

- Node.js.
- Express.js.
- A modular API structure separating conversation management, file processing, transcription, and LLM interaction.

Database

- MongoDB.
- Use a MongoDB Node.js driver or Mongoose, choosing one approach and explaining why.

MongoDB is preferred because the application uses Node.js, and conversation messages and structured LLM outputs map naturally to document-oriented storage.

LLM

Use an existing API-accessible LLM rather than training a model.

Gemini is a possible initial choice, but the plan should select a suitable model and SDK based on the requirements, cost, supported features, and ease of integration.

Keep API keys in server-side environment variables. Never expose provider API keys in frontend code.

Audio transcription

Use an existing speech-to-text API or model, selected according to cost, accepted file formats, size limits, transcription quality, and integration complexity.

The implementation plan should identify one recommended option rather than leaving several competing options unresolved.

PDF processing

Use an appropriate server-side PDF text extraction library.

OCR for image-only or scanned PDFs is optional and can be postponed.

Persistence and file storage

Use MongoDB for conversation data and message history.

For the student implementation, files may be stored in a persistent server-side uploads directory with validated filenames and access controls. Store file metadata and references in MongoDB rather than storing large audio and PDF binaries directly in ordinary conversation documents.

Do not rely on temporary operating-system paths that might disappear unexpectedly. Explain how uploaded file retention will work.

Development environment

Assume local development on a typical Windows computer using VS Code and Node.js. Include required software and installation prerequisites in the plan.

Do not introduce Docker, cloud deployment, microservices, vector databases, or complicated infrastructure unless a specific requirement justifies them.

---

8. MongoDB Data Model

The implementation plan must propose and explain a practical database schema.

A suggested initial design uses two collections: `conversations` and `messages`.

8.1 Conversations collection

Each conversation represents one meeting or related discussion.

Suggested fields:

- `_id`: Unique conversation identifier.
- `clientId`: Anonymous browser identifier.
- `title`: Sidebar title.
- `sourceType`: Type of original source, such as audio, TXT, PDF, or pasted text.
- `originalFilename`: Original filename when a file was uploaded.
- `sourceText`: Extracted transcript or meeting notes needed for later follow-up questions, subject to size and retention considerations.
- `sourceFileReference`: Optional reference to the stored original file.
- `createdAt`: Conversation creation timestamp.
- `updatedAt`: Most recent activity timestamp.

The plan may adjust these fields if there is a better design, but it must explain where the meeting context is stored and how it remains accessible after a conversation is reopened.

If transcript size or separation of responsibilities makes it preferable, source documents may be placed in a separate collection. Do not add unnecessary collections without a concrete reason.

8.2 Messages collection

Each message should be stored separately.

Suggested fields:

- `_id`: Unique message identifier.
- `conversationId`: Reference to the parent conversation.
- `role`: `user` or `assistant`.
- `content`: Exact message content to display.
- `messageType`: Optional classification, such as ordinary text, meeting summary, or error.
- `structuredData`: Optional structured meeting-minutes data.
- `attachments`: Optional file metadata associated with the message.
- `createdAt`: Message timestamp.

Create the necessary indexes, especially for retrieving messages by `conversationId` and ordering them by creation time.

Explain how message ordering will remain stable if timestamps are identical or messages are created close together.

8.3 Data persistence rules

The plan must explain:

- When a conversation record is created.
- When a user message is saved.
- When an assistant response is saved.
- What happens if the LLM request fails after the user message has been saved.
- How previous conversations are loaded.
- How a deleted conversation's messages and associated files are handled.
- How the application verifies that a requested conversation belongs to the supplied anonymous client identifier.

The exact user and assistant messages must survive page reloads and browser restarts, subject to available storage and the application's retention policy.

Do not use browser `localStorage` as the sole permanent store for full conversation history. It should be used for the anonymous client identifier and may optionally cache lightweight UI state; MongoDB should hold the authoritative conversation history.

---

9. Backend API Requirements

Propose a practical REST API. The following endpoint names are illustrative and may be refined.

- `GET /api/conversations` — List conversations belonging to the requesting anonymous browser.
- `POST /api/conversations` — Create a conversation when needed.
- `GET /api/conversations/:id/messages` — Retrieve messages for a selected conversation.
- `POST /api/conversations/:id/messages` — Submit text or an eligible follow-up message.
- `POST /api/conversations/:id/files` — Upload a meeting file, if this separation is useful.
- `DELETE /api/conversations/:id` — Delete a conversation and its associated data, if implemented.

The plan should decide whether file uploading and message submission should be combined or separated. Prefer the simplest architecture that cleanly supports progress feedback and error handling.

For every endpoint, specify:

- Purpose.
- Expected request parameters and body.
- Response structure.
- Validation rules.
- Relevant status codes.
- How conversation ownership is checked.
- Likely errors.

Do not assume that sending a request to the LLM alone is sufficient. The backend must persist messages, retain source context, validate requests, and return predictable responses to the frontend.

---

10. LLM Integration and Prompt Design

The LLM API must be called by the backend.

The application should maintain a predefined instruction for meeting-minute generation. Users should not need to create their own prompts.

The meeting-analysis prompt should define the expected information categories, formatting rules, and evidence constraints.

Follow-up questions should use a separate or appropriately adapted prompt that instructs the model to:

- Answer based on the meeting content.
- Consider prior relevant messages in the active conversation.
- Avoid inventing unsupported details.
- Distinguish explicit facts from interpretations.
- State when the supplied source does not contain enough information.
- Avoid confusing the current meeting with other conversations.

Use structured output for meeting minutes where supported. Validate the returned data before rendering it.

A failed, malformed, or incomplete response should not crash the application. Provide a useful error message and a reasonable retry option.

The plan should discuss token limits and whether the extracted text must be truncated, summarized, chunked, or retrieved selectively for longer meetings.

For the initial version, prefer sending the entire extracted meeting text when it comfortably fits the chosen model's context limits. Do not implement RAG or a vector database by default. Introduce retrieval or chunking only when practical transcript sizes require it.

Do not present API-based LLM integration as model training. The project's technical contribution is building and evaluating a working application around an existing model.

---

11. Processing Workflow

The plan must define the workflow for each source type.

Audio workflow

1. User uploads an audio recording.
2. Backend validates the file and checks configured limits.

3. A speech-to-text service transcribes the recording.
4. Backend validates the transcription result.
5. Extracted text is retained as the source context.
6. The LLM generates structured meeting minutes.
7. User and assistant messages are persisted.
8. Frontend displays the results and permits follow-up questions.

TXT workflow

1. Receive the uploaded file.
2. Validate its extension, MIME type where available, and size.
3. Read and normalize its text.
4. Reject empty or unreadable content.
5. Pass the content into the meeting-analysis workflow.

PDF workflow

1. Receive and validate the PDF.
2. Extract the text on the backend.
3. Detect when the PDF is empty or contains no usable text.
4. Display an understandable error when extraction fails.
5. Optionally recommend OCR support for scanned PDFs rather than silently returning an empty summary.
6. Pass the extracted text into the meeting-analysis workflow.

Pasted text workflow

1. User enters meeting notes or a transcript.
2. Frontend submits the text to the backend.
3. Backend validates input and creates or updates the active conversation.
4. The LLM generates meeting minutes.
5. The application saves and displays the result.

Follow-up workflow

1. User submits a new question in an existing conversation.
2. Backend retrieves the conversation and its source context.
3. Backend retrieves the relevant recent messages.
4. The LLM generates an answer grounded in the source.
5. Both the user's message and assistant's reply are persisted.
6. Frontend updates the current chat without losing earlier messages.

Conversation restoration workflow

1. User clicks an earlier conversation.
2. Frontend requests its messages from the backend.
3. Backend checks access using the anonymous client identifier.
4. Backend returns saved messages ordered consistently.
5. Frontend renders the exact saved content.
6. The source context remains available for future questions.

---

12. Security, Reliability, and Limitations

The plan must account for the following without turning the project into an unnecessarily complex security system:

- Keep API keys on the backend.
- Validate file extensions, MIME types where applicable, and upload sizes.
- Avoid trusting user-supplied filenames or paths.
- Use safe file-storage paths and clean up files when appropriate.
- Validate and sanitize user input.
- Restrict access to conversations based on their stored anonymous client identifier.
- Enforce upload limits and reasonable request timeouts.
- Return useful errors for provider outages, transcription failures, invalid files, and rate limits.
- Avoid logging complete confidential transcripts or secrets unnecessarily.

- Explain how the application handles uploaded meeting files and source text after deletion.
- Avoid treating anonymous browser IDs as secure authentication.
- Distinguish model-generated content from verified facts and permit user review.
- Consider prompt injection inside uploaded transcripts. Meeting content should be treated as data to analyze, not instructions that can override the application's governing prompt.
- Prevent an LLM response from being rendered as executable HTML.
- Avoid unnecessary duplication of large transcripts or files.

For a college demonstration, a simple local file-storage implementation is acceptable, but the plan must acknowledge that files stored on a local server may not persist across redeployments or work across multiple server instances.

---

13. Scope Boundaries

The project must remain manageable for a student team.

Essential features

1. ChatGPT-style two-section interface.
2. New Chat and previous-conversation sidebar.
3. Direct text input.
4. TXT and selectable-text PDF support.
5. Uploaded audio transcription.
6. Automatic structured meeting minutes.
7. Context-aware follow-up questions.
8. MongoDB persistence for conversations and messages.
9. Exact restoration of previously saved conversation history.
10. Anonymous browser identity, with no login.
11. Basic export.

12. File validation and meaningful error handling.

Optional enhancements

Prioritize these only after the essential workflow works:

- OCR for scanned PDFs.
- Editing generated meeting minutes.
- PDF export.
- Editable conversation titles.
- More advanced conversation search.
- Speaker identification or diarization.
- Meeting timestamps attached to extracted statements.
- Retrieval or chunking for very long transcripts.
- Cloud file storage.
- Authentication or synchronization across devices.
- Live microphone recording and real-time transcription.
- Calendar integrations, reminders, and email notifications.

Do not implement unrelated features such as scheduling systems, task-management dashboards, resume analysis, job matching, or an independent study assistant.

---

14. Testing and Evaluation

The project must demonstrate that the application works beyond one successful example.

Design tests covering:

- A short pasted meeting transcript.
- A longer transcript with multiple decisions and participants.
- A TXT transcript.
- A PDF containing selectable text.
- A PDF that contains no extractable text.

- A valid audio recording.
- An unsupported or excessively large file.
- A meeting where no deadlines are specified.
- A meeting that contains suggestions but no confirmed decisions.
- A follow-up question whose answer exists in the source.
- A follow-up question whose answer is absent from the source.
- Returning to a previous conversation after refreshing the browser.
- Verifying that historical responses are not regenerated.
- Starting a new meeting without mixing in the previous meeting's context.
- An LLM API error or timeout.
- An attempted request to access a conversation belonging to a different anonymous identifier.

Assess output quality using correctness of extracted decisions, completeness of action items, preservation of important details, unsupported claims, response usefulness, and processing time.

Use a small set of test transcripts with manually prepared expected results to evaluate the LLM output.

The application must not be declared reliable merely because it generates plausible-looking responses.

---

15. Expected Deliverables From You

Using all the requirements above, produce a detailed implementation plan with the following sections, in this order:

A. Requirements review

Summarize the requirements and distinguish essential features from optional ones.

B. Technical architecture

Describe how React, Express, MongoDB, the LLM API, speech-to-text, and PDF extraction fit together. Include a simple architecture or data-flow diagram.

C. Technology decisions

Choose a recommended LLM provider and speech-to-text option. Explain why each choice fits this student project, along with likely cost and usage considerations. Identify any important limits that should be checked before implementation.

D. System and data-flow diagrams

Show how audio, PDFs, TXT files, pasted text, initial analysis, follow-up questions, and conversation restoration move through the application.

E. Database schema

Define the collections, fields, relationships, indexes, and persistence lifecycle.

F. API design

Specify the REST endpoints, request and response structures, status codes, error cases, and conversation-access checks.

G. Prompt and LLM architecture

Explain initial meeting analysis, structured output validation, follow-up question handling, context retention, and handling of long inputs.

H. Frontend design

Describe the layout, sidebar behavior, chat area, file-upload flow, loading states, error states, new-chat behavior, and previous-conversation restoration.

I. Project folder structure

Propose a practical React and Express directory structure. Explain the responsibility of important files without generating all the source code.

J. Implementation phases

Break the work into sequential phases. Each phase should state its objective, tasks, dependencies, expected output, and acceptance criteria.

Prioritize building a working text-based workflow with saved conversation history first, then add PDF input, follow-up questions, and audio transcription in a sensible order. The plan should explain whether a different ordering is more appropriate.

K. Testing strategy

Provide a practical test checklist for unit tests, API tests, end-to-end flows, and LLM output evaluation.

L. Environment setup

List required software, packages, environment variables, API keys, local setup steps, and instructions for avoiding exposure of secrets. Do not expose real credentials.

M. Common implementation mistakes

Explain likely problems involving asynchronous processing, database persistence, duplicate messages, context mixing, invalid LLM output, uploaded files, anonymous identity, and restored chats.

N. Final acceptance checklist

Define the minimum conditions required for the application to be considered complete.

O. Progress tracking

Present a task-by-task checklist that a student can use while implementing the project.

---

16. Instructions for Your Planning Process

Before writing the plan, reason about the dependencies between features so that the implementation sequence minimizes rework.

Do not start generating application source code. The immediate deliverable is the plan.

Do not overengineer the design. Prefer understandable, modular code and a small number of dependencies.

Make decisions rather than listing many alternatives without a recommendation. Where a choice genuinely depends on provider capabilities, identify what must be checked.

Clearly label assumptions, optional work, and anything requiring a paid API. Avoid promising that a third-party service is free or supports a particular format without verifying its current terms and limits.

If a detail is not specified but can reasonably be assumed, state the assumption and continue. Ask clarification questions only when a missing decision materially blocks a sound implementation plan.

The final plan should be detailed enough that a beginner can implement MeetBrief AI phase by phase without having to redesign the project's architecture halfway through development.